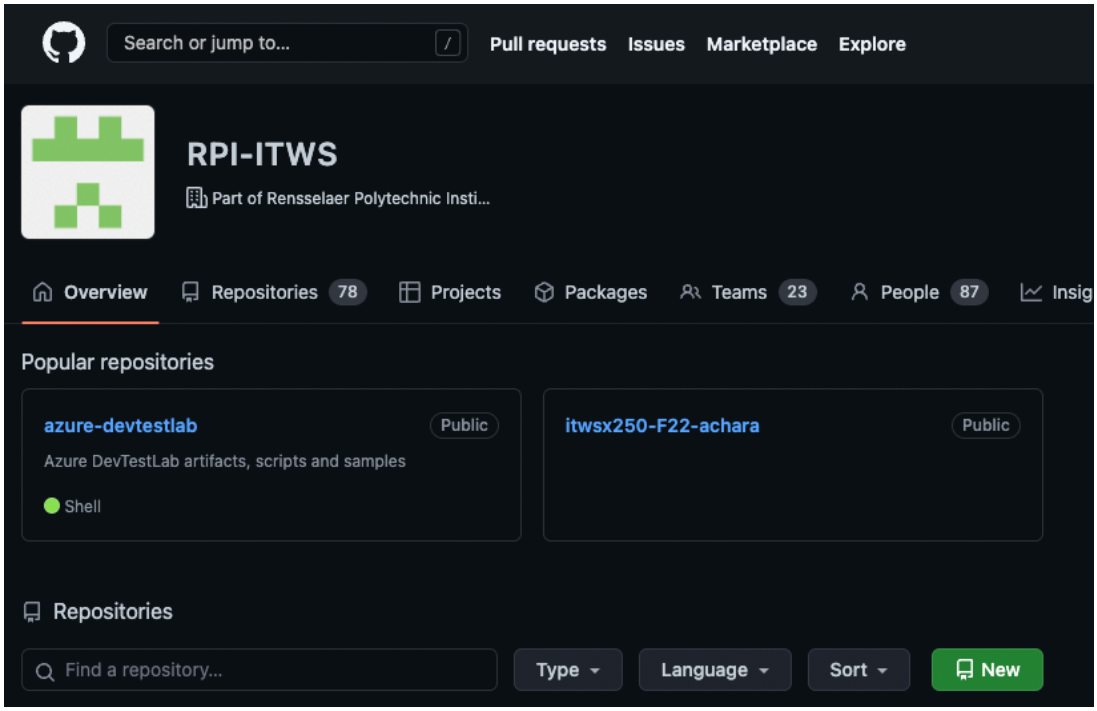
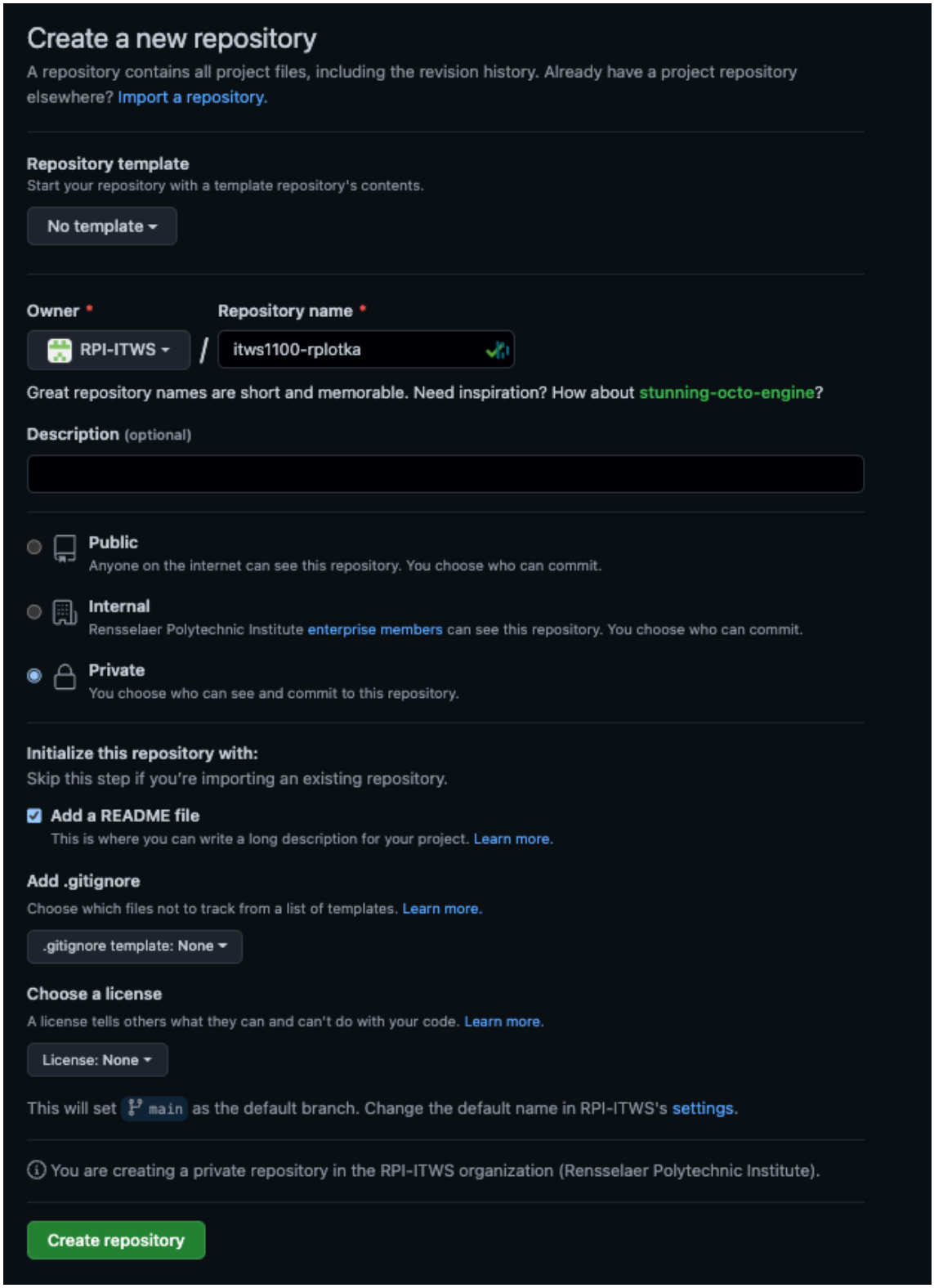
GitHub and Azure

1 – GitHub

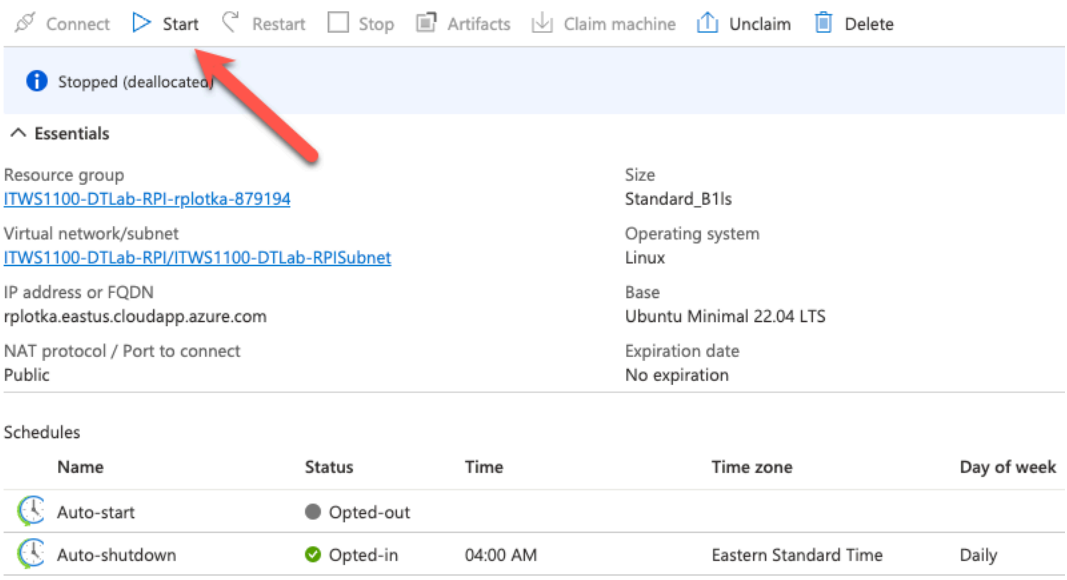
Let’s log on to GitHub and create a repository for our classwork




Create a new repository
A repository contains all project files, including the revision history. Already have a project repository elsewhere? Import a repository.
Repository template
Start your repository with a template repository's contents.
No template
Owner *
Repository name *
RPI-ITWS
/
itws1100-rplotka
Great repository names are short and memorable. Need inspiration? How about stunning-octo-engine?
Description (optional)
Public
Anyone on the internet can see this repository. You choose who can commit.
Internal
Rensselaer Polytechnic Institute enterprise members can see this repository. You choose who can commit.
Private
You choose who can see and commit to this repository.
Initialize this repository with:
Skip this step if you're importing an existing repository.
Add a README file
This is where you can write a long description for your project. Learn more.
Add .gitignore
Choose which files not to track from a list of templates. Learn more.
.gitignore template: None
Choose a license
A license tells others what they can and can't do with your code. Learn more.
License: None
This will set main as the default branch. Change the default name in RPI-ITWS's settings.
You are creating a private repository in the RPI-ITWS organization (Rensselaer Polytechnic Institute).
Create repository

Now let’s log on to our servers

2 – start the instance at portal.azure.com



3a – In order to have git 'talk' to our repository, we need to create a key pair – this will allow us to authenticate with GitHub to clone our repo.

Let's create a key so we can authenticate to GitHub
https://docs.github.com/en/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent

From a Terminal session on our local computers, SSH into your server:
ssh uid@FQDN

Now that we're on our instances,

Let's create a key pair using the ssh-keygen command. Type:

sudo -u www-data ssh-keygen -t ed25519 -C "[yourRCSid]@rpi.edu"

When the system asks for a filename, accept the default.

(It will then ask for a passphrase (ie password)
(Enter a password – don't forget it!)

Verify that the file was created by typing

ls -la /var/www/.ssh

You should see at least one file with the name

id_ed25519.pub

Now let's start the system's security agent service. Type:

eval "$(ssh-agent -s)"

We now need to add the *private* key to our system's list of known users. Type

sudo -u www-data ssh-agent ssh-add /var/www/.ssh/id_ed25519

(You will be asked for the passphrase) Type it in

3b - Now we need to add the *public* key to GitHub
https://docs.github.com/en/github/authenticating-to-github/adding-a-new-ssh-key-to-your-github-account

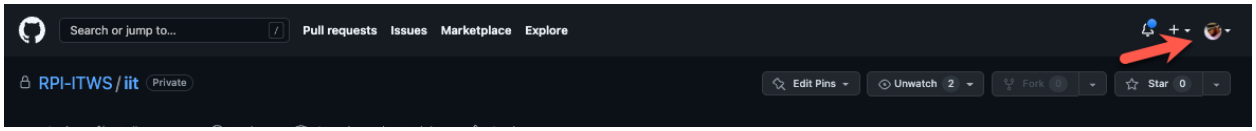
let's copy the public key to our clipboard. Type

cat /var/www/.ssh/id_ed25519.pub

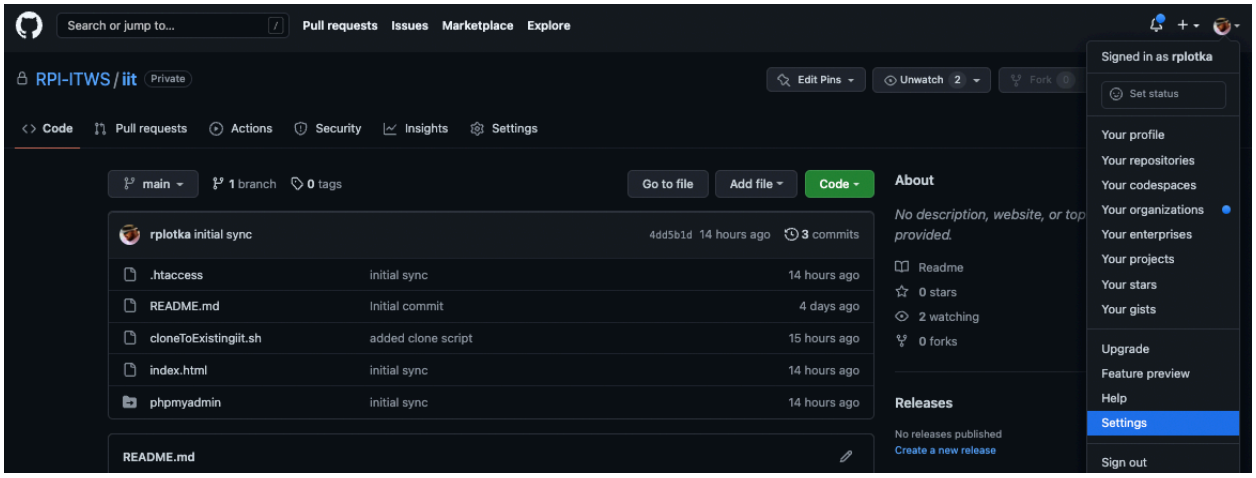
Highlight the line of text starting with ssh-ed25519… and copy it

```
rplotka@rplotka:/var/www/html/iit$ cat ~/.ssh/id_ed25519.pub
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIAzX3onMSUSwCf/l6ilYO2qEmTzhDd7DlgcTK7YygMAS rplotka@rpi.edu
```

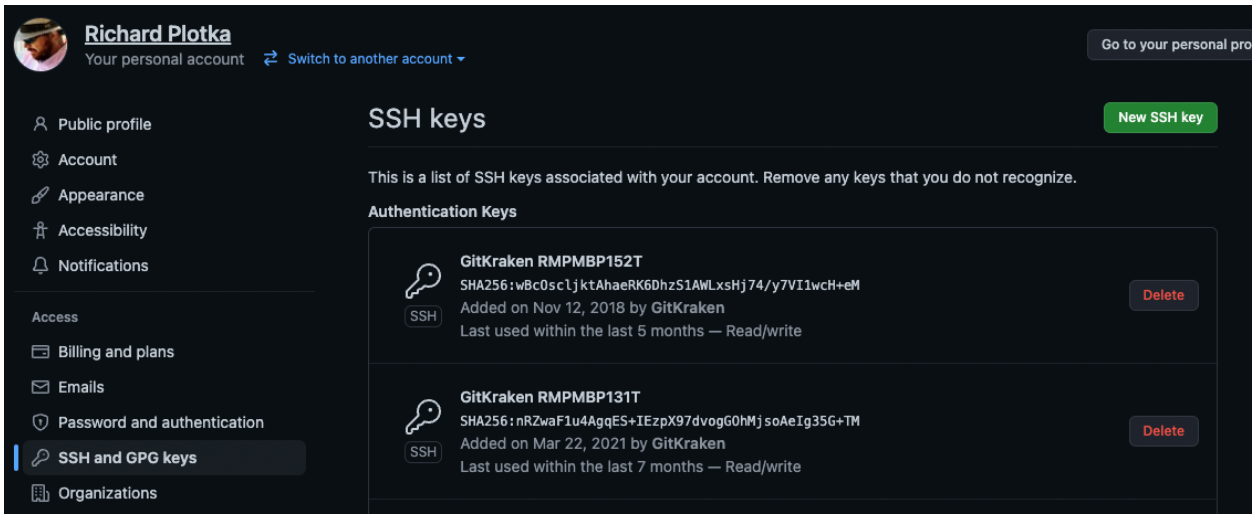
Now go to your GitHub account in your browser, and click on your profile icon



Select Settings

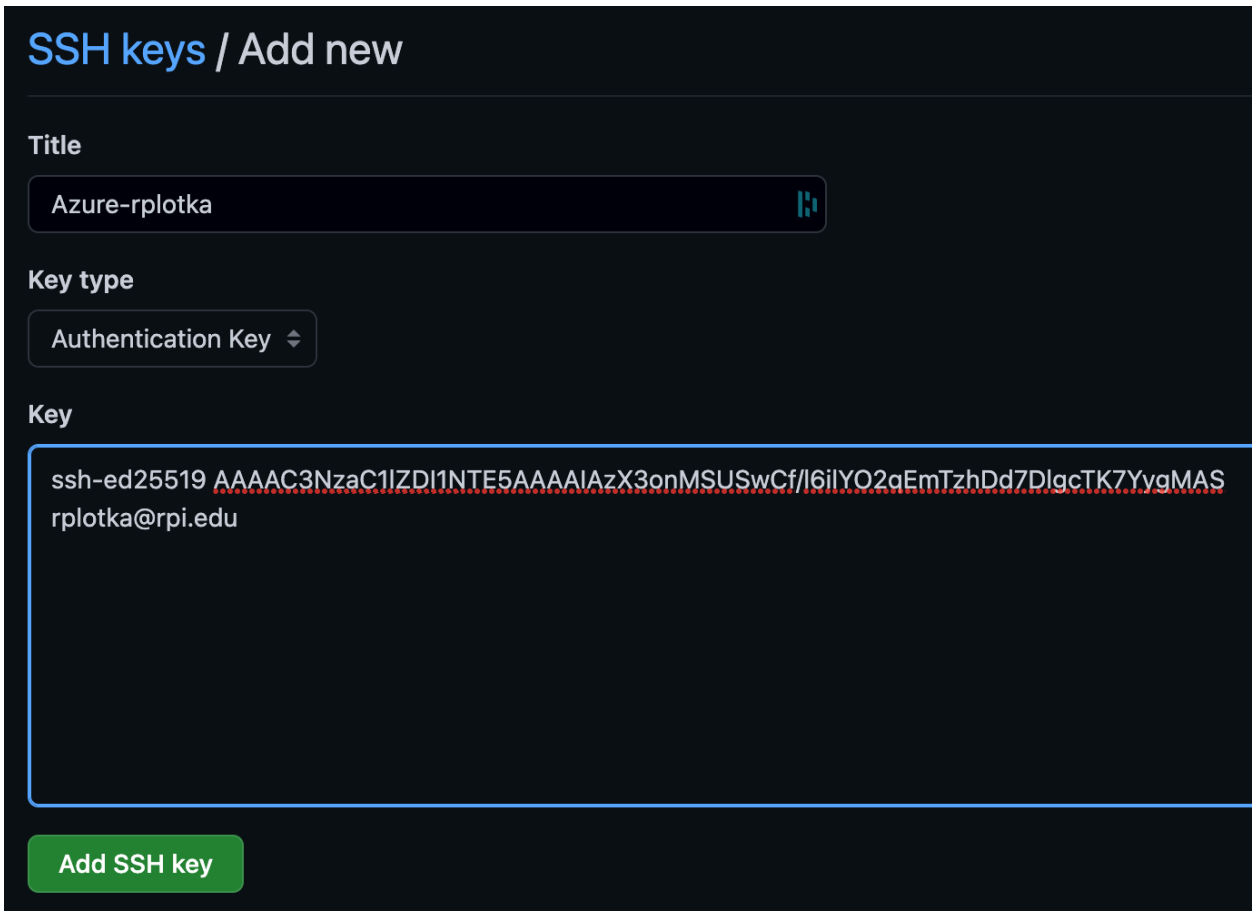


Select SSH and GPD keys



Click New 

Give it a name, and paste the key into the Key field



(You may be asked for your PW. If so, enter it)

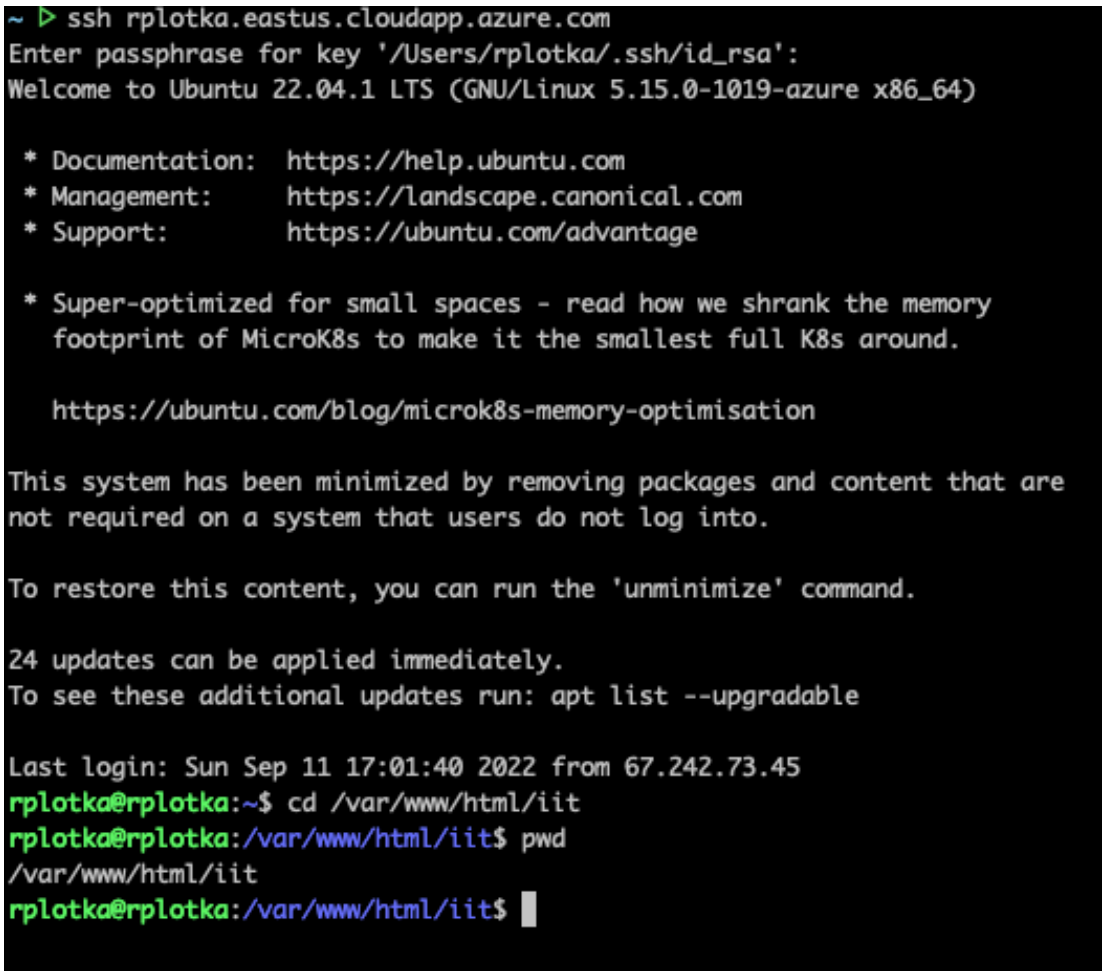
4 – We have now told the git program on our Azure webservers how to ‘talk’ to GitHub. So let’s connect the repo on GitHub with the the iit folder on our servers. Let’s first clone the repo.

Let’s go back to our session on our servers, then move into your iit folder. Type

cd /var/www/html/iit

Check to see where our directory pointer is located. Type

pwd



OK, let's execute the 'git clone' command to connect our folder to our repo.

Type the following, replacing [yourRepoName] with the name of your repo (ITWS1100-[yourRCSid]). Type the following commands:

We want to connect our repo and our iit folder. We have a small problem – GitHub will not clone into an existing folder…So, we are going to trick it with 4 commands.
NOTE: for #1, we must use the user www-data, since our key to GitHub uses this ID. For 2-4, you should not need the www-data user parameter. They 'should' only be needed when issuing a command to your GitHub repo.

1. Clone the repo into a temporary folder named tmp
    a. sudo -u www-data git clone git@github.com:RPI-ITWS/[yourRepoName].git tmp
2. Move (mv=move) the .git folder (which contains the history that sits in GitHub) into our iit folder
    a. sudo mv tmp/.git .
3. Delete (rm=remove) the temporary folder
    a. sudo rm -rf tmp
4. Reset gits pointer to compare git in our iit folder with the 'known' inventory on GitHub
    a. sudo -u www-data git reset

Might need to enter if told by linux
git config –global –add safe.directory /var/www/html/iit

Let's see what happened by checking the status of git on our machines. type

git status

(You will see something similar to the below because we have the files we already created in Lab 1, and the README.md file we just created when we created our repos)

```
rplotka@rplotka:/var/www/html/iit$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    README.md
        deleted:    cloneToExistingiit.sh

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .htaccess
        index.html
        phpmyadmin
```

This tells us that we have files on GitHub that are not on our server and files on our server that are not on GitHub.

We want to get both sides synchronized.

So:
The system thinks we deleted files locally – so let's restore the index, so let's put them into the inventory. Type

sudo -u www-data git restore .

Let's copy (pull) the files down from the online repo

sudo -u www-data git pull

Let's check our folder. Type

ls -la

```
rplotka@rplotka:/var/www/html/iit$ ls -la
total 28
drwxr-xr-x 3 rplotka iit     4096 Sep 11 03:57 .
drwxr-xr-x 3 root    root    4096 Aug 27 15:08 ..
drwxrwxr-x 8 rplotka rplotka 4096 Sep 11 03:57 .git
-rw-rw-r-- 1 rplotka rplotka  100 Sep 11 03:57 .htaccess
-rw-rw-r-- 1 rplotka rplotka    5 Sep 11 03:57 README.md
-rw-rw-r-- 1 rplotka rplotka  132 Sep 11 03:57 cloneToExistingiit.sh
-rw-rw-r-- 1 rplotka rplotka   65 Sep 11 03:57 index.html
lrwxrwxrwx 1 rplotka rplotka   21 Sep 11 03:57 phpmyadmin -> /usr/share/phpmyadmin
```

We can see that we now have both the files from the repo and our local files from before on our system. However, git does not yet know that we want it to track our local files, so…

Let's add our local files. Type

sudo -u www-data git add .

Now check git. Type

git status

```
rplotka@rplotka:/var/www/html/iit$ git push
Everything up-to-date
rplotka@rplotka:/var/www/html/iit$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        new file:   .htaccess
        new file:   index.html
        new file:   phpmyadmin
```

This tells us that we are tracking everything and that our new (files not yet sent to the server) files are waiting to be committed to the local repo

So, let's commit them into the local repo. (the -m flag allows us to put in the required commit message right from the command line) Type

NOTE: If you cut/paste the command below – you will need to retype the single quotes

sudo -u www-data git commit -m ‘initial sync’

and

git status

```
rplotka@rplotka:/var/www/html/iit$ git commit -m 'initial sync'
[main 4dd5b1d] initial sync
 3 files changed, 12 insertions(+)
 create mode 100644 .htaccess
 create mode 100644 index.html
 create mode 120000 phpmyadmin
rplotka@rplotka:/var/www/html/iit$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
```

This tells us that our local repo is up to date but that we are out of sync ‘ahead’ of (more current than) our online version – so let’s fix that by sending our local repo up to GitHub and updating our online repo

Send the local files to GitHub. Type

sudo -u www-data git push

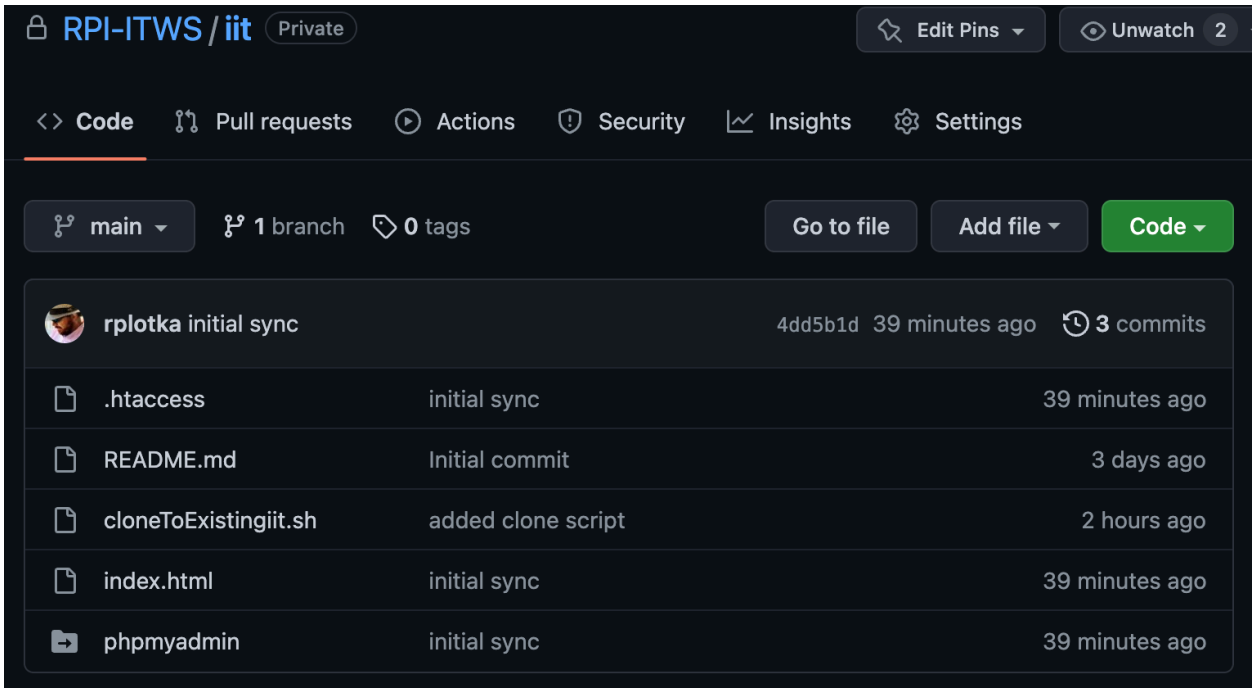
and

git status

```
rplotka@rplotka:/var/www/html/iit$ git push
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Compressing objects: 100% (4/4), done.
Writing objects: 100% (5/5), 568 bytes | 568.00 KiB/s, done.
Total 5 (delta 0), reused 0 (delta 0), pack-reused 0
To github.com:RPI-ITWS/iit.git
   65752d1..4dd5b1d  main -> main
rplotka@rplotka:/var/www/html/iit$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
rplotka@rplotka:/var/www/html/iit$
```

Let's check our GitHub repo in our bowsers

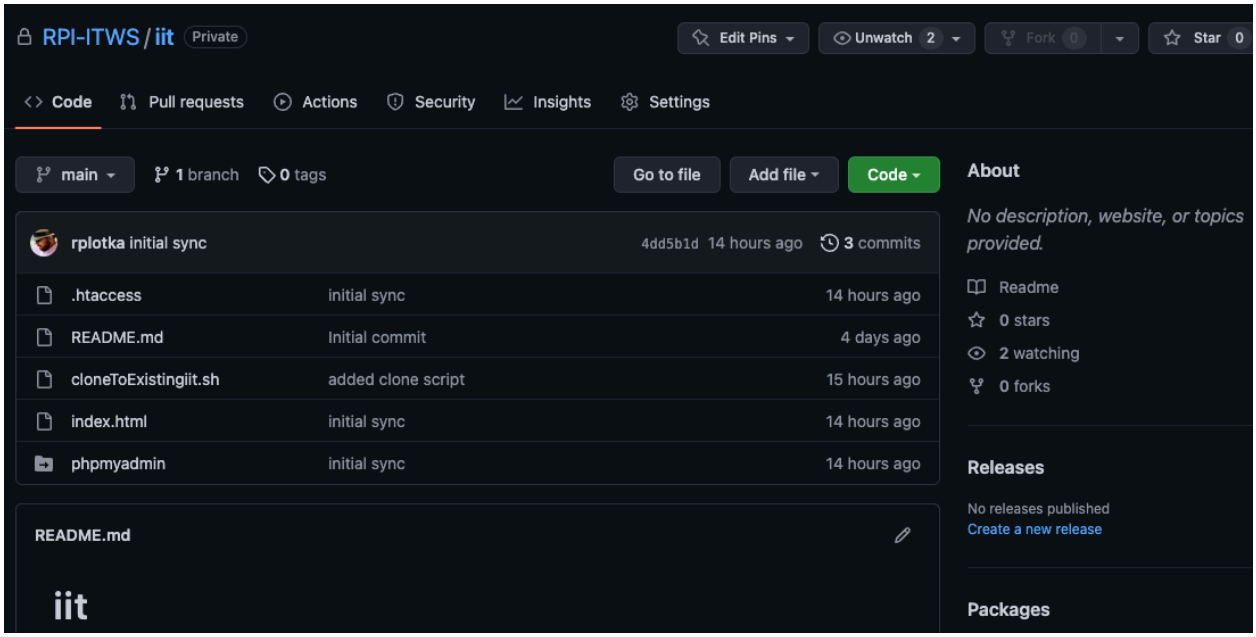


Our GitHub repo and Web Server's iit folder are now connected and synced! Well Done!

We now need to connect our GitHub repos to our local machines for development. This will be much easier since we will start fresh.
5 – Clone the repo to our local machine

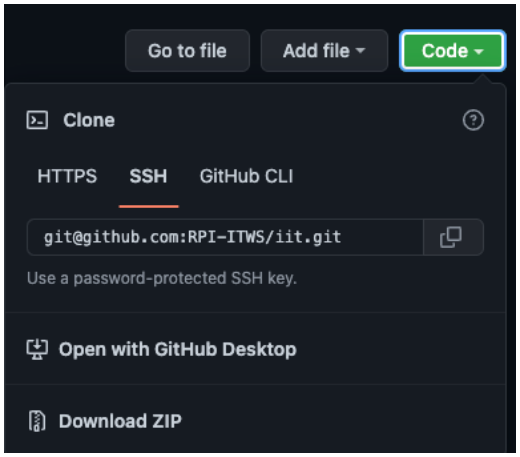
Using our Browser – eg Chrome, lets open our GitHub Repo

(In this example, my repo is called iit – yours should be called itws1100-[yourRCSid])



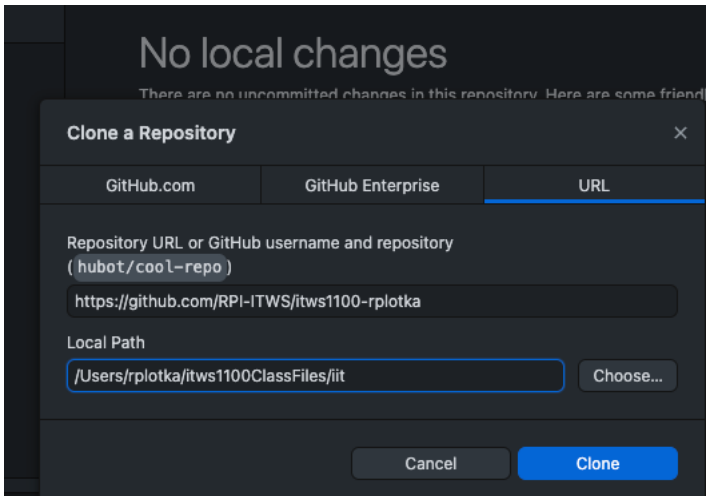
Click the  button

Select ‘Open with GitHub Desktop



Github Desktop will open

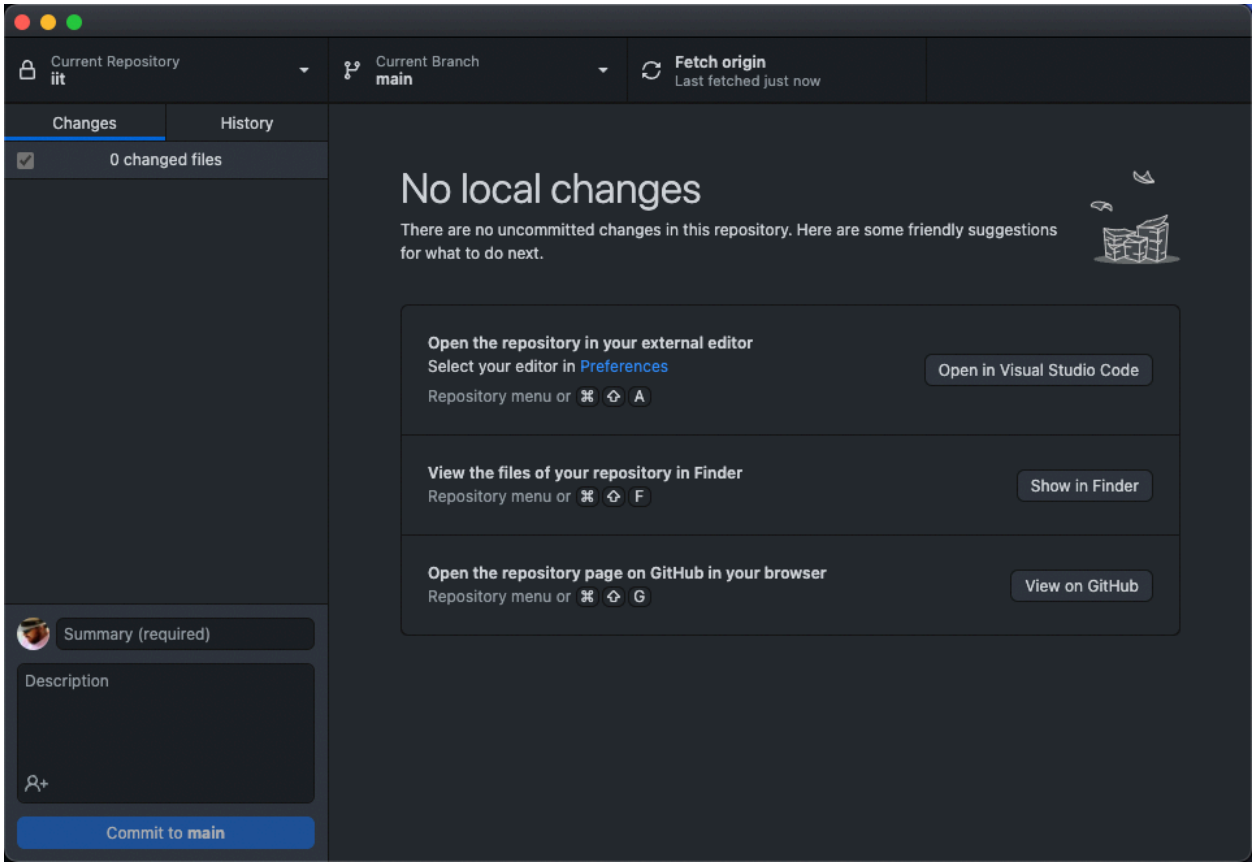
(You may get a prompt asking for permission – allow it)
You need to select a folder on your local machine for our class work. In the example here, I have a folder called ITWS1100-ClassFiles. I am going to have GitHub put my repository there and name it iit (for clarity) you may use any name you like, including the default.



Click the 'Clone' button

GitHub Desktop will create the folder and copy the files into it.

When done, GitHub Desktop will give you a screen allowing you to do a few things.

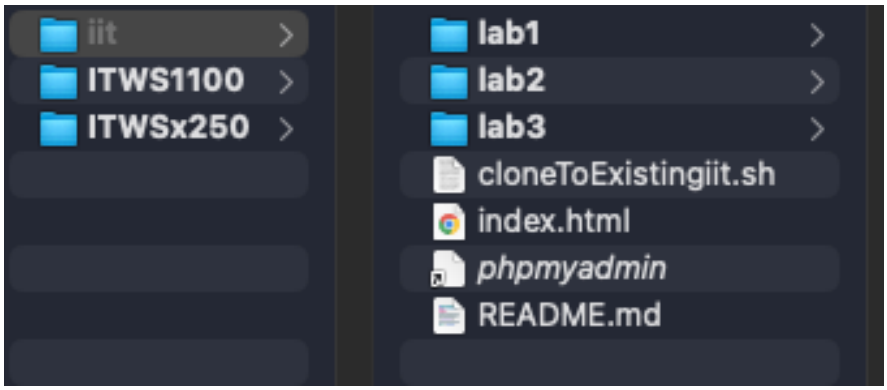


If you click the View the Files link – your system's file explorer will open to that folder/directory

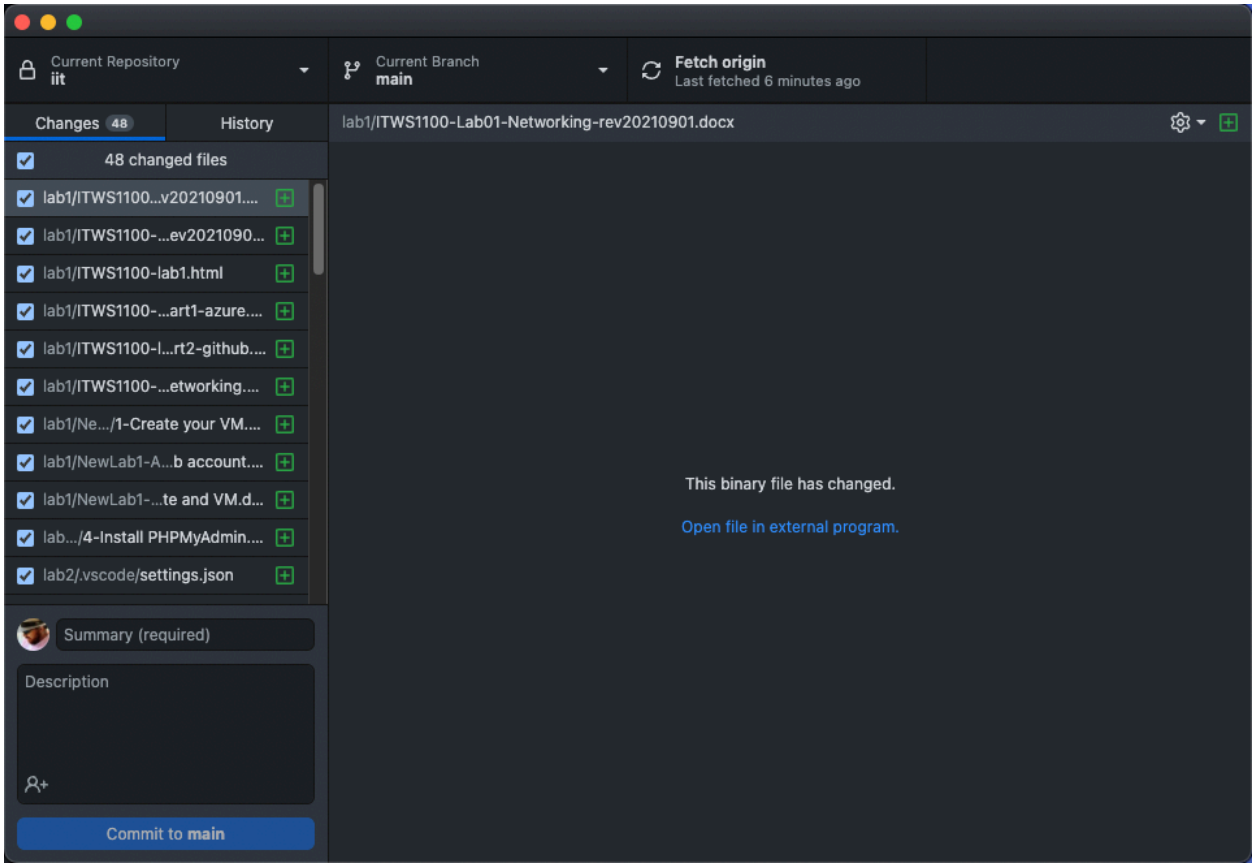
If you click 'Open with Visual Studio Code', VS Code will open with all of your files

But first – we need to copy our lab 1, and lab 2 folders and make a lab 3 folder

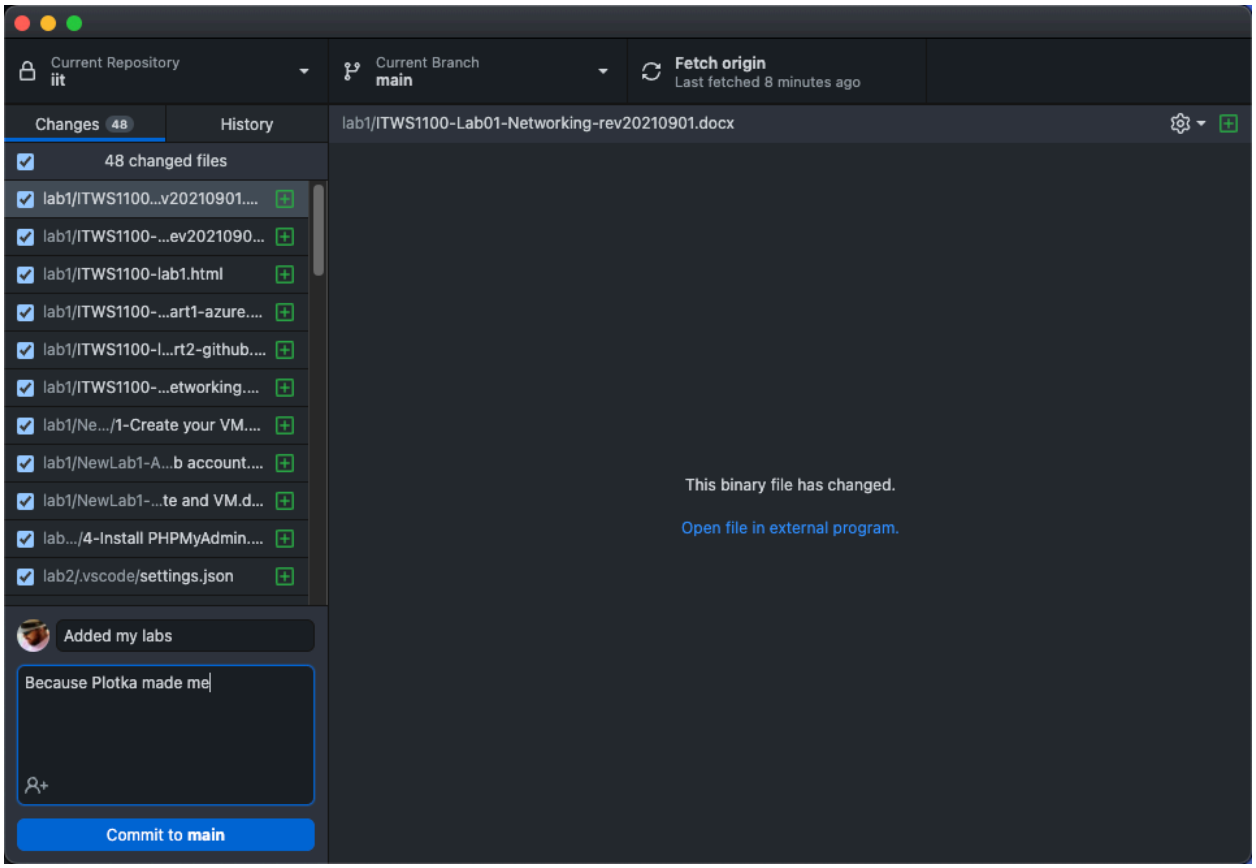
Using our normal File Explorer (Finder on Mac), lets copy our lab1, and lab 2 folders into the new iit folder, and create a lab 3 folder if you don't already have one.



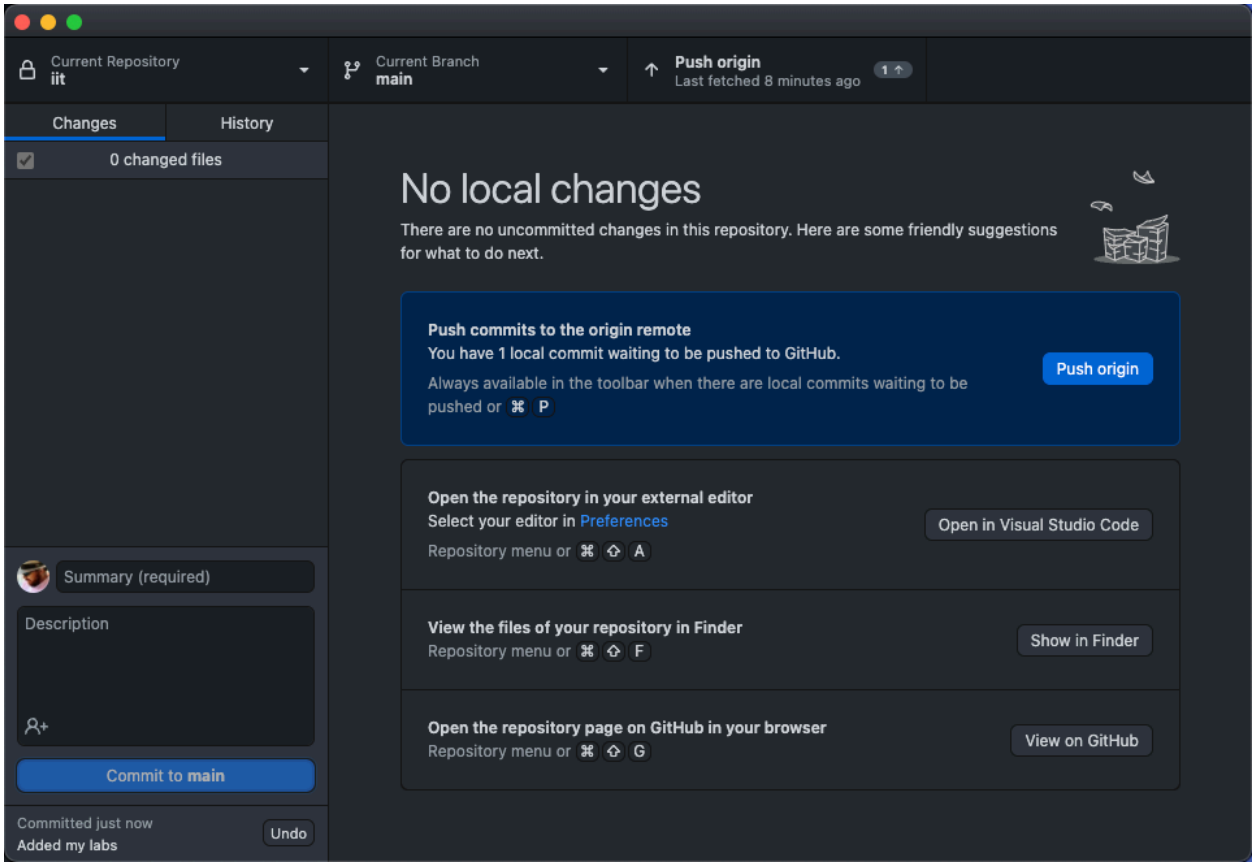
Now let's check out GitHub Desktop – notice the screen has changed, reflecting all our changes on the left. (just like a git status from the command line)



Let's commit these changes. Type a message into the Summary field (remember the -m message before?). Then enter some detail in the Description field – something meaningful to remind you why you are doing this. Click Commit

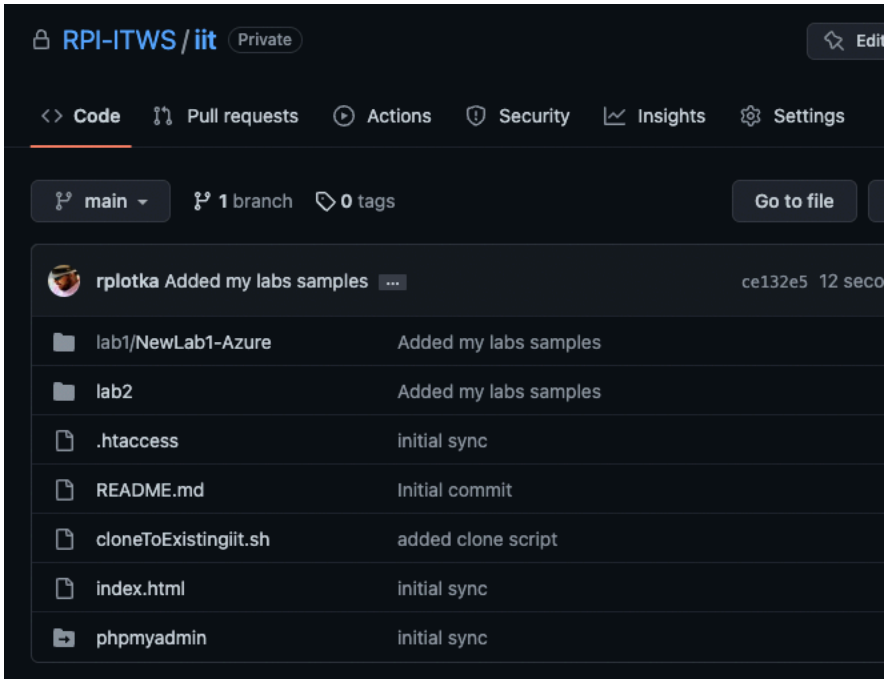


After clicking commit, we get the following screen



Click Push Origin – this sends our local repo to GitHub. Exactly like the 'git push' from the command line above

Check your results on GitHub in the Browser – you will see all of your files….



Now – lets go put these onto our web servers…

7 – Lets bring our changes down from GitHub to our Cloud Servers on Azure
(Note: I am going to start giving you instructions and letting you determine the appropriate command)

Make sure your server is running:
SSH into your server
CD into your iit folder
Let's 'fetch' the current status of our repo, type

sudo -u www-data git fetch

```
rplotka@rplotka:/var/www/html/iit$ git fetch
Enter passphrase for key '/home/rplotka/.ssh/id_ed25519':
remote: Enumerating objects: 13, done.
remote: Counting objects: 100% (13/13), done.
remote: Compressing objects: 100% (8/8), done.
remote: Total 12 (delta 1), reused 12 (delta 1), pack-reused 0
Unpacking objects: 100% (12/12), 4.59 MiB | 14.63 MiB/s, done.
From github.com:RPI-ITWS/iit
   4dd5b1d..ce132e5  main       -> origin/main
```

Now let's check the status

git status

```
rplotka@rplotka:/var/www/html/iit$ git status
On branch main
Your branch is behind 'origin/main' by 1 commit, and can be fast-forwarded.
  (use "git pull" to update your local branch)

nothing to commit, working tree clean
```

Let's bring down our changes, type

sudo -u www-data git pull

```
rplotka@rplotka:/var/www/html/iit$ git pull
Enter passphrase for key '/home/rplotka/.ssh/id_ed25519':
Updating 4dd5b1d..ce132e5
Fast-forward
 lab1/NewLab1-Azure/1-Create your VM.docx            | Bin 0 -> 532228 bytes
 lab1/NewLab1-Azure/2-Setup your GutHub account.docx | Bin 0 -> 2220082 bytes
 lab1/NewLab1-Azure/3-Access your website and VM.docx | Bin 0 -> 1061949 bytes
 lab1/NewLab1-Azure/4-Install PHPMyAdmin.docx        | Bin 0 -> 1309790 bytes
 lab2/.vscode/settings.json                          |   3 +++
 lab2/Intro ITWS-S18-Lab2-HTML-ExampleInProgress.html |  65 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
 6 files changed, 68 insertions(+)
 create mode 100755 lab1/NewLab1-Azure/1-Create your VM.docx
 create mode 100755 lab1/NewLab1-Azure/2-Setup your GutHub account.docx
 create mode 100755 lab1/NewLab1-Azure/3-Access your website and VM.docx
 create mode 100755 lab1/NewLab1-Azure/4-Install PHPMyAdmin.docx
 create mode 100755 lab2/.vscode/settings.json
 create mode 100755 lab2/Intro ITWS-S18-Lab2-HTML-ExampleInProgress.html
```

check to make sure it worked

git status

and then

ls -la

```
rplotka@rplotka:/var/www/html/iit$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
rplotka@rplotka:/var/www/html/iit$ ls -la
total 36
drwxr-xr-x 5 rplotka iit     4096 Sep 11 18:38 .
drwxr-xr-x 3 root    root    4096 Aug 27 15:08 ..
drwxrwxr-x 8 rplotka rplotka 4096 Sep 11 18:38 .git
-rw-rw-r-- 1 rplotka rplotka  100 Sep 11 03:57 .htaccess
-rw-rw-r-- 1 rplotka rplotka    5 Sep 11 03:57 README.md
-rw-rw-r-- 1 rplotka rplotka  132 Sep 11 03:57 cloneToExistingiit.sh
-rw-rw-r-- 1 rplotka rplotka   65 Sep 11 03:57 index.html
drwxrwxr-x 3 rplotka rplotka 4096 Sep 11 18:38 lab1
drwxrwxr-x 3 rplotka rplotka 4096 Sep 11 18:38 lab2
lrwxrwxrwx 1 rplotka rplotka   21 Sep 11 03:57 phpmyadmin -> /usr/share/phpmyadmin
rplotka@rplotka:/var/www/html/iit$
```

OK – Now everything is synced!!!

Check it out..

You have now setup a GitHub repo and connected it with a working folder on your local computer for editing and with your servers for deployment.

Going forward, we will work on our local machines, commit and push our changes to our Repos and then pull them down to our Azure Web Server Instances to deploy/publish.

Welcome to DevOps!

Final Note: Notice that the procedures we follow from the command line on our webservers are the same as the ones we follow through the interface in GitHub Desktop. GitHub Desktop is just showing us prettier screens and executing the appropriate command for us when we click the pretty buttons. If you open a Terminal session on your local machines and navigate (set your file pointer using CD) to your iit folder, and type a git status, you will see something familiar.